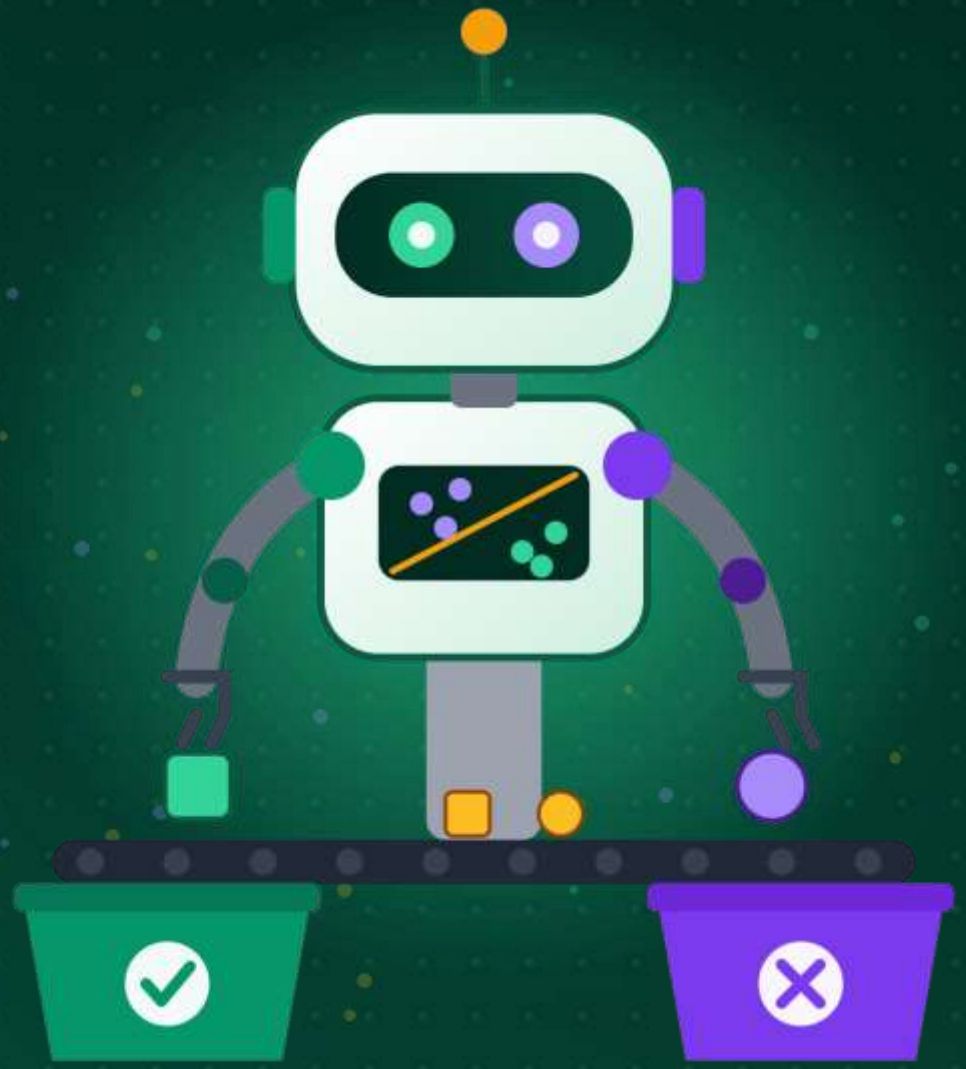
VSP TECHVERSE
An Institute of AI & Robotics

Classification Models for AI & Robotics

Logistic Regression · Support Vector Machine · K-Nearest Neighbours

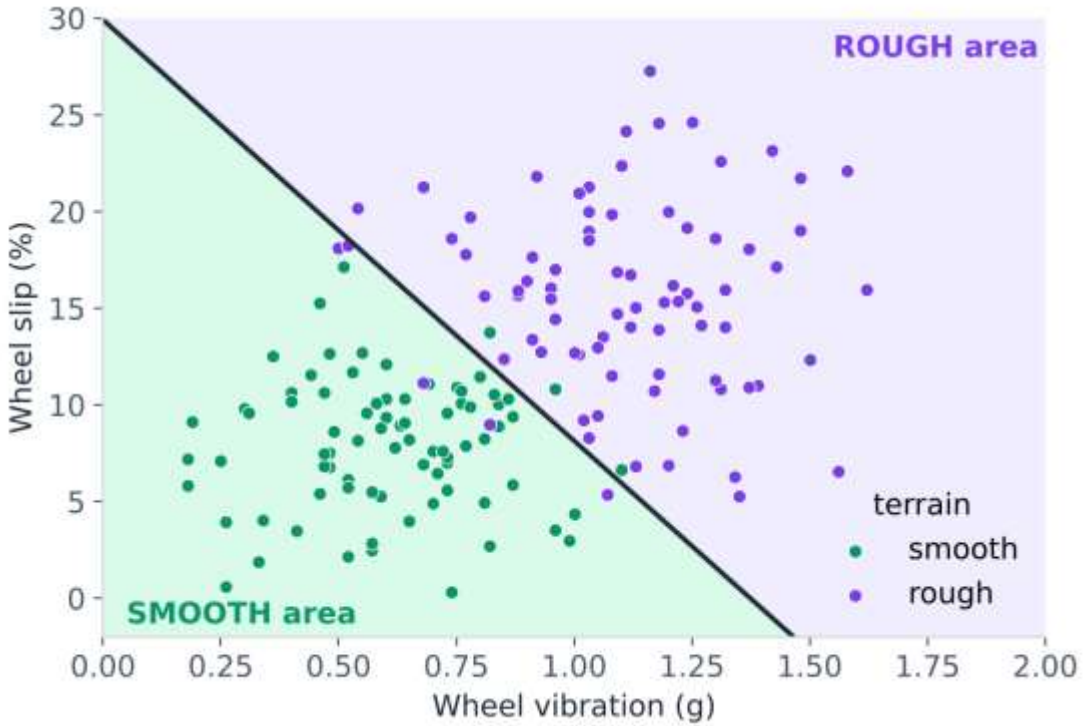


VSP TECHVERSE
An Institute of AI & Robotics

What is Classification?

Classification is supervised learning where the model predicts a category (a class label) instead of a number.

- **Binary:** two classes, e.g. grasp success / fail
- **Multi-class:** three or more, e.g. bolt / nut / washer
- **Decision boundary:** the line or curve that separates the classes
- **Regression vs classification:** "how much?" vs "which one?"



ROBOTICS EXAMPLE A rover reads wheel vibration and slip, then decides: is the ground smooth or rough?

Our Generated Robot Datasets

Each model gets its own synthetic dataset, generated with NumPy so you can see clearly how the model behaves.

Logistic Regression

robot_grasp_data.csv

200 rows

FEATURES (X)
grip_force_n
object_weight_kg

TARGET (y)
success: 1 / 0

Will the gripper hold the object?

Linear SVM

robot_terrain_data.csv

160 rows

FEATURES (X)
vibration_g
wheel_slip_pct

TARGET (y)
terrain: smooth / rough

What surface is the rover on?

SVM with RBF kernel

robot_zone_data.csv

220 rows

FEATURES (X)
x_m
y_m

TARGET (y)
zone: danger / safe

Is this spot inside the danger circle?

K-Nearest Neighbours

robot_parts_data.csv

150 rows

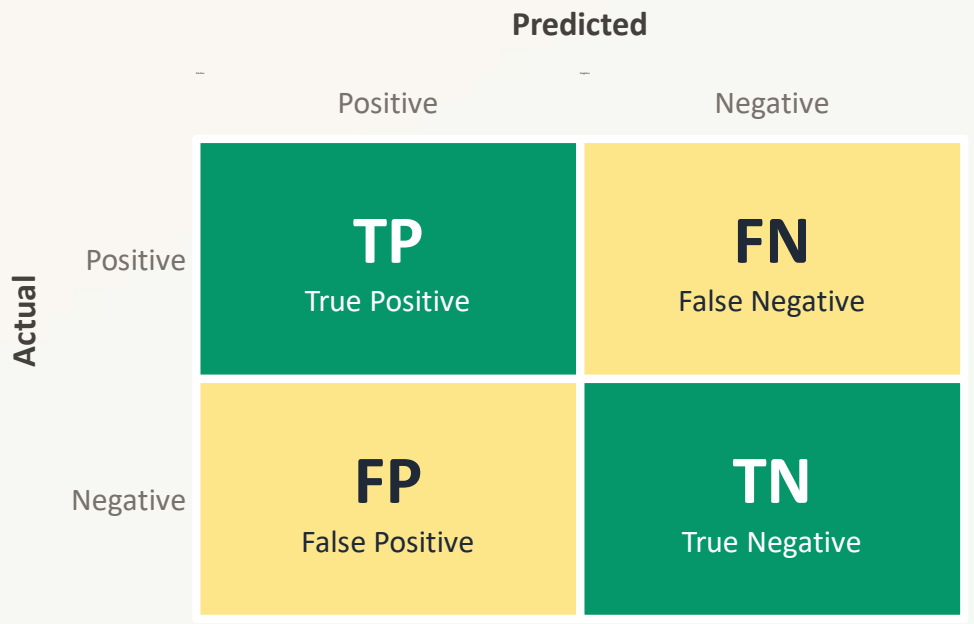
FEATURES (X)
weight_g
length_mm

TARGET (y)
part: bolt / nut / washer

Which part is on the conveyor?

Classification Evaluation Metrics

CONFUSION MATRIX



Green = correct predictions · Yellow = mistakes

ROBOTICS EXAMPLE A false negative (missed failed grasp) can drop a part; a false positive wastes a retry.

Accuracy $(TP + TN) / All$

Share of all predictions that are correct

Precision $TP / (TP + FP)$

Of the predicted positives, how many were right?

Recall $TP / (TP + FN)$

Of the real positives, how many did we find?

F1 score $2 \cdot P \cdot R / (P + R)$

Balance of precision and recall

MODEL 1

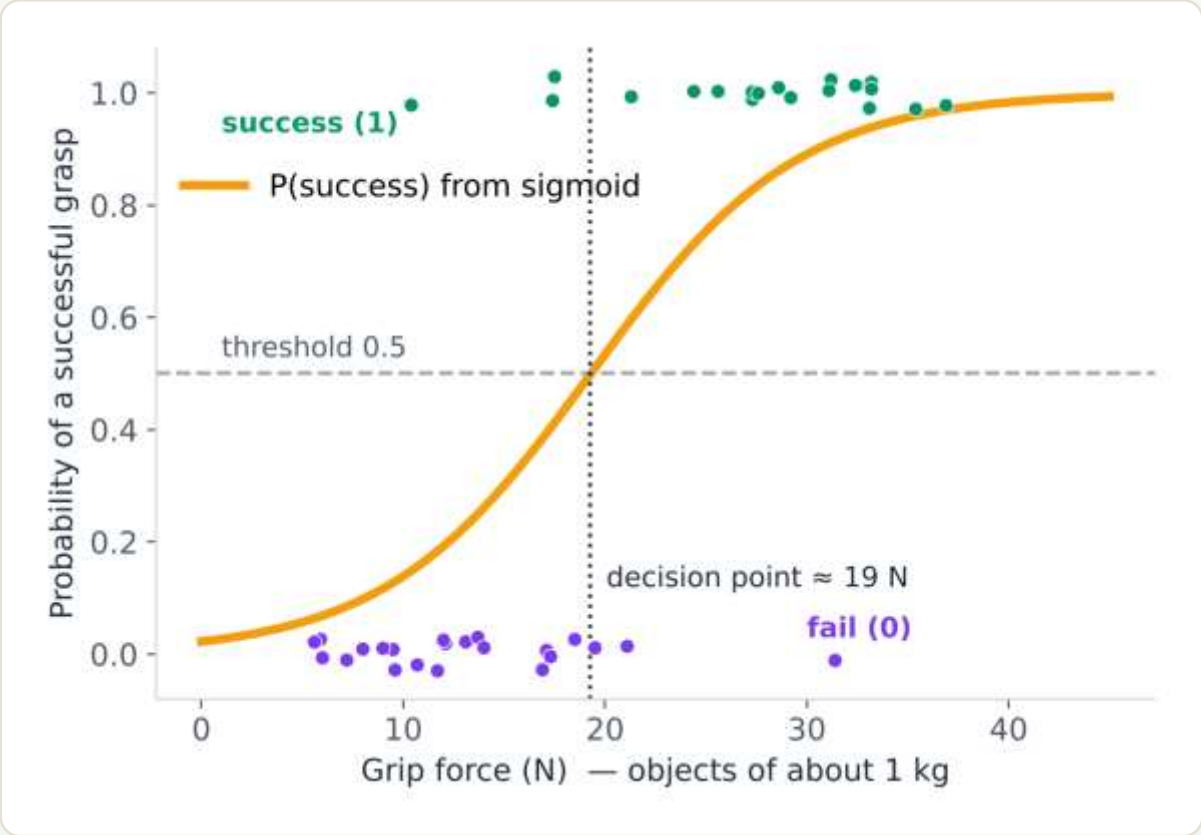
Logistic Regression

Despite its name, logistic regression is a classification model. It computes a weighted sum of the inputs, then squashes it into a probability between 0 and 1 with the sigmoid function.

SIGMOID

$$z = b_0 + b_1x_1 + b_2x_2$$
$$\sigma(z) = 1 / (1 + e^{-z})$$

- **Output:** probability of class 1, e.g. P(success)
- **Threshold:** P ≥ 0.5 → class 1, otherwise class 0
- **Boundary:** a straight line where P = 0.5
- **Training:** finds weights that best match the labels



ROBOTICS EXAMPLE For a 1 kg object, grip force above about 19 N gives a probability above 0.5.

Logistic Regression: Grasp Success

logistic.py

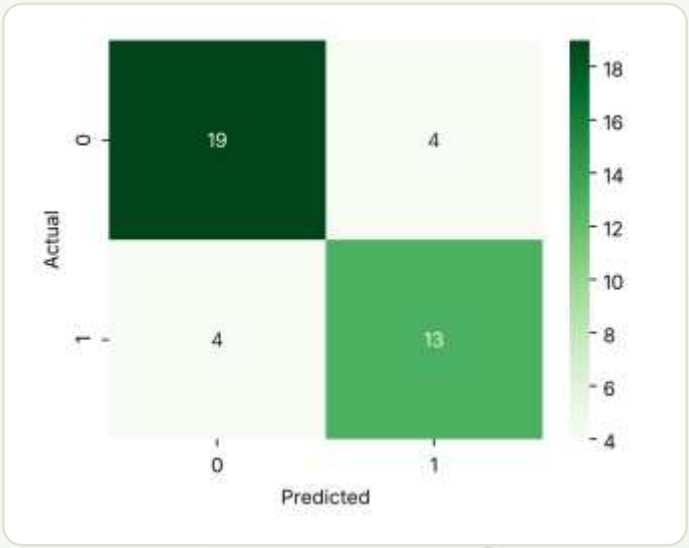
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

df = pd.read_csv("robot_grasp_data.csv")
X, y = df[["grip_force_n", "object_weight_kg"]], df["success"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy :", round(metrics.accuracy_score(y_test, y_pred), 2))
print("Precision:", round(metrics.precision_score(y_test, y_pred), 2))
print("Recall   :", round(metrics.recall_score(y_test, y_pred), 2))
print("F1 score :", round(metrics.f1_score(y_test, y_pred), 2))

cm = metrics.confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, cmap="Greens")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()
```

CONFUSION MATRIX (TEST DATA)



OUTPUT

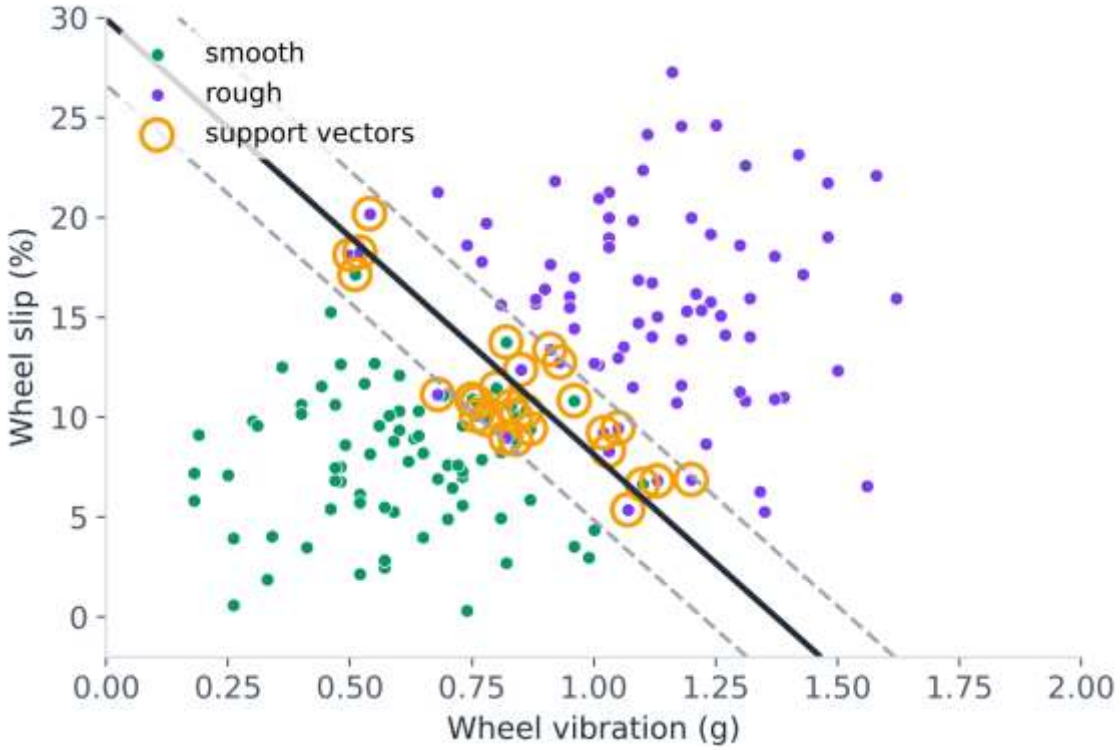
```
Accuracy : 0.8
Precision: 0.76
Recall   : 0.76
F1 score : 0.76
```

The diagonal of the heatmap holds the correct predictions. 80% of the test grasps are classified correctly.

Support Vector Machine (SVM)

An SVM finds the boundary (hyperplane) that separates the classes with the widest possible gap, called the margin.

- **Hyperplane:** the decision boundary; a line in 2D
- **Margin:** the gap between the boundary and the nearest points
- **Support vectors:** the closest points; they alone define the boundary
- **C parameter:** small C allows a wider, softer margin; large C fits training data tightly
- **Scaling:** always scale features first (StandardScaler)



ROBOTICS EXAMPLE A wide margin keeps the rover's terrain decision stable even with noisy sensor readings.

Linear SVM: Terrain Detection

svm_linear.py

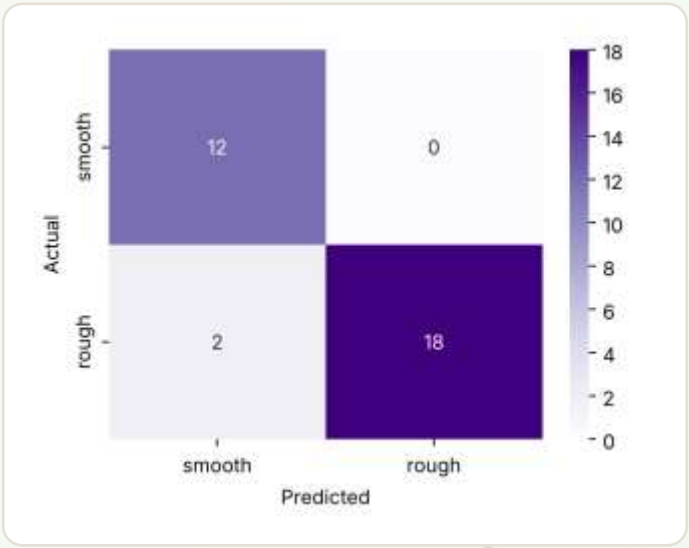
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn import metrics

df = pd.read_csv("robot_terrain_data.csv")
X, y = df[["vibration_g", "wheel_slip_pct"]], df["terrain"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = make_pipeline(StandardScaler(), SVC(kernel="linear"))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", round(metrics.accuracy_score(y_test, y_pred), 2))
print("F1 score:", round(metrics.f1_score(y_test, y_pred,
                                          pos_label="rough"), 2))

labels = ["smooth", "rough"]
cm = metrics.confusion_matrix(y_test, y_pred, labels=labels)
sns.heatmap(cm, annot=True, cmap="Purples",
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()
```

CONFUSION MATRIX (TEST DATA)



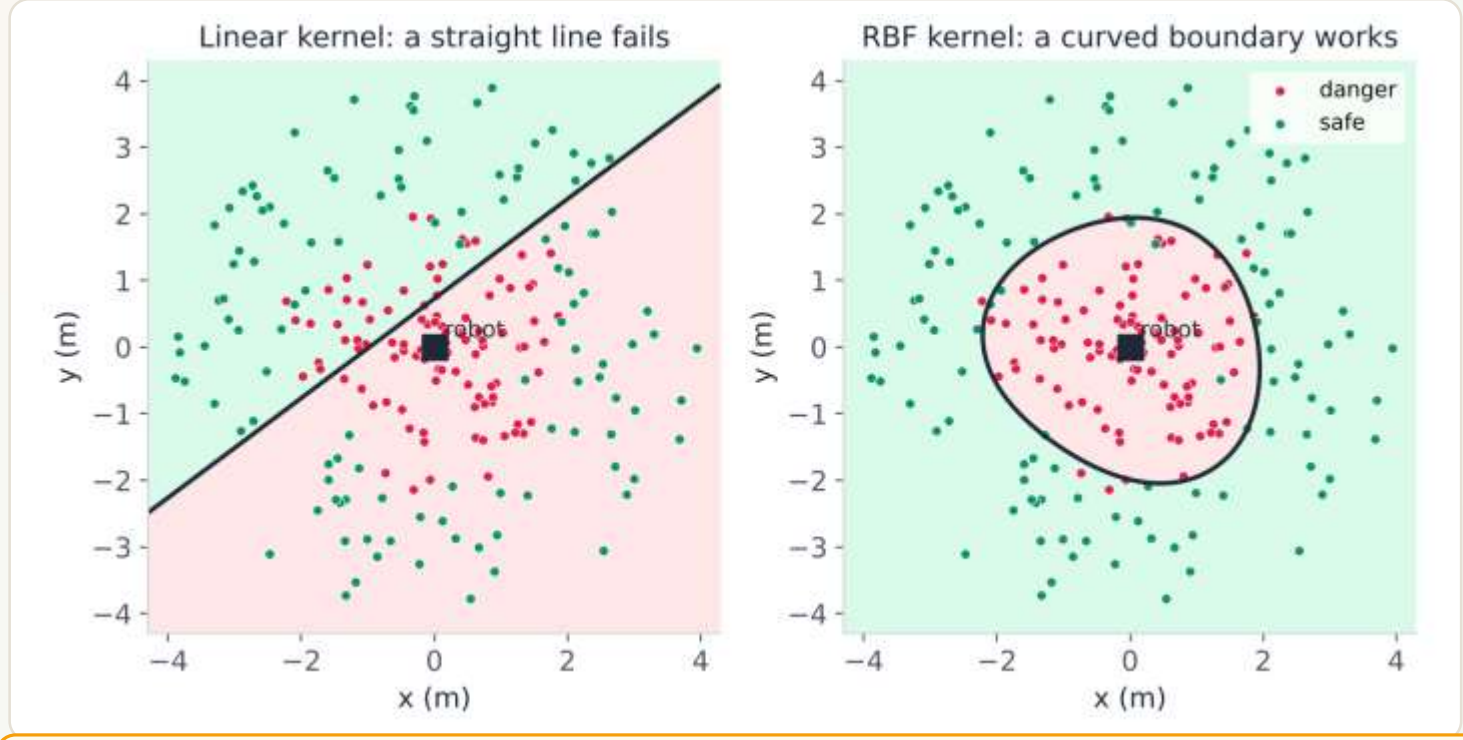
OUTPUT

```
Accuracy: 0.94
F1 score: 0.95
```

Only a few terrain readings are misclassified: the two classes are almost linearly separable.

The Kernel Trick: Non-linear SVM

Some data cannot be split by a straight line. A kernel lets SVM draw curved boundaries by measuring similarity in a higher-dimensional space.



COMMON KERNELS

`kernel="linear"`
Straight line. Fast; use when classes are linearly separable.

`kernel="rbf"`
Smooth curved regions. A good default for non-linear data.

`kernel="poly"`
Polynomial curves of a chosen degree.

ROBOTICS EXAMPLE The danger zone around a robot arm is a circle: no straight line can separate it, but the RBF kernel can.

SVM with RBF Kernel: Danger Zone

svm_rbf.py

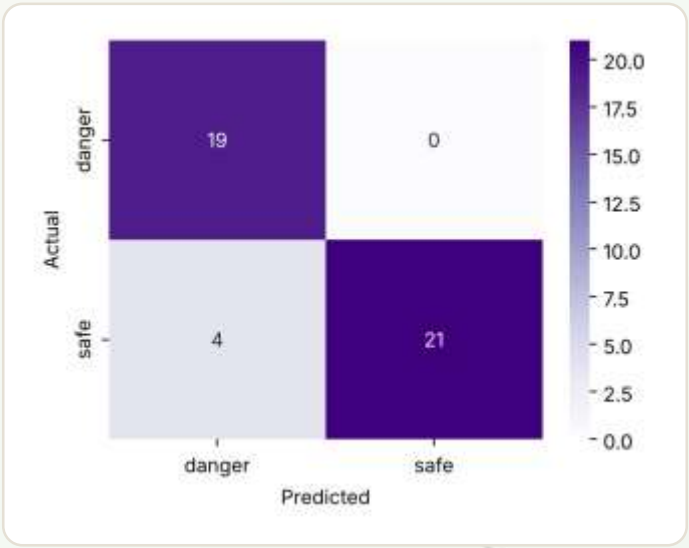
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn import metrics

df = pd.read_csv("robot_zone_data.csv")
X, y = df[["x_m", "y_m"]], df["zone"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

for k in ["linear", "rbf"]:
    model = make_pipeline(StandardScaler(), SVC(kernel=k))
    model.fit(X_train, y_train)
    print(k, "accuracy:", round(model.score(X_test, y_test), 2))

y_pred = model.predict(X_test)    # the last model is rbf
labels = ["danger", "safe"]
cm = metrics.confusion_matrix(y_test, y_pred, labels=labels)
sns.heatmap(cm, annot=True, cmap="Purples",
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()
```

CONFUSION MATRIX (TEST DATA)



OUTPUT

```
linear accuracy: 0.48
rbf accuracy: 0.91
```

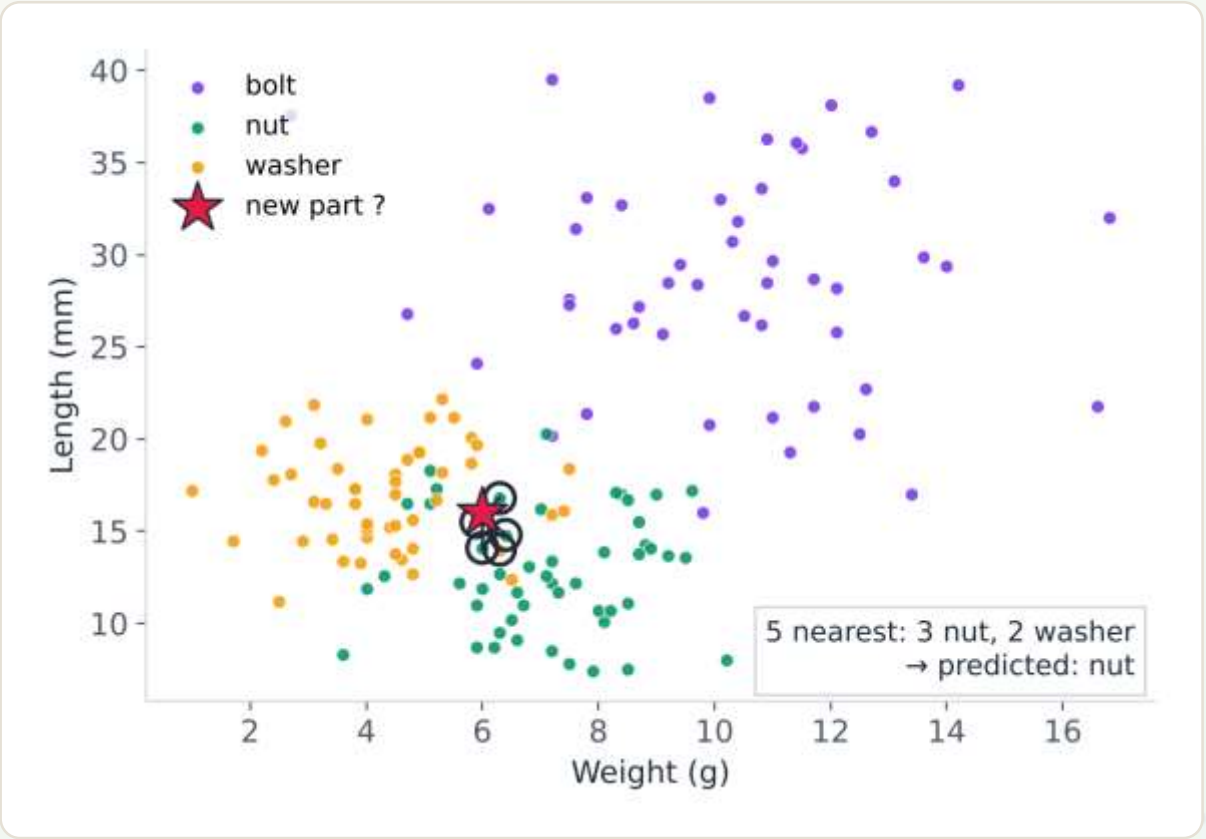
Same data, same split: the linear kernel is no better than a coin flip (0.48), while RBF reaches 0.91.

MODEL 3

K-Nearest Neighbours (KNN)

KNN has no training formula. To classify a new point it looks at the k closest training points and takes a majority vote.

1. Choose k (e.g. 5)
2. Measure distance to every training point
3. Pick the k nearest neighbours
4. Majority vote → predicted class



EUCLIDEAN DISTANCE

$$d = \sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2]}$$

Use an odd k for two classes to avoid ties, and always scale features so one unit does not dominate the distance.

KNN: Sorting Parts on a Conveyor

knn.py

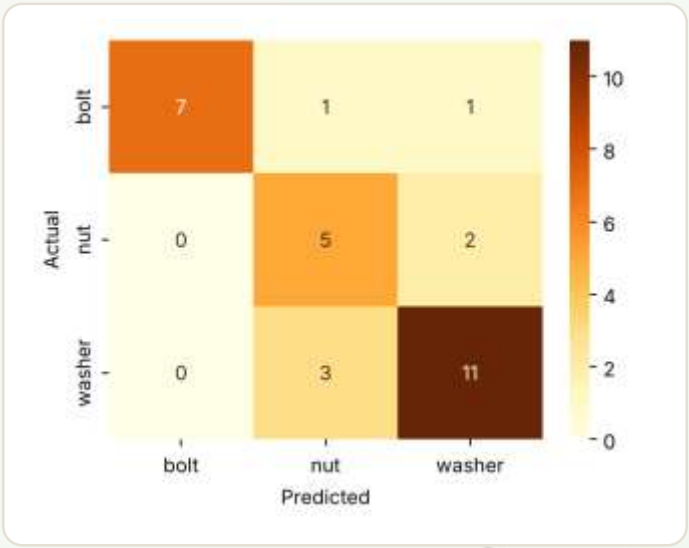
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn import metrics

df = pd.read_csv("robot_parts_data.csv")
X, y = df[["weight_g", "length_mm"]], df["part"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", round(metrics.accuracy_score(y_test, y_pred), 2))
print("F1 (macro):", round(metrics.f1_score(y_test, y_pred,
                                            average="macro"), 2))

names = model.classes_
cm = metrics.confusion_matrix(y_test, y_pred, labels=names)
sns.heatmap(cm, annot=True, cmap="YlOrBr",
            xticklabels=names, yticklabels=names)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()
```

CONFUSION MATRIX (TEST DATA)



OUTPUT

```
Accuracy: 0.77
F1 (macro): 0.76
```

Most mistakes are between nuts and washers, whose weights and lengths overlap. Bolts are easy to spot.

COMPARISON

Advantages & Disadvantages

Logistic Regression

ADVANTAGES

- Simple, fast and easy to explain
- Gives probabilities, not just labels
- Works well on small datasets

DISADVANTAGES

- Only a linear boundary
- Struggles with complex patterns
- Sensitive to outliers

Support Vector Machine

ADVANTAGES

- Strong accuracy with a clear margin
- Kernels handle non-linear data
- Works well with many features

DISADVANTAGES

- Slow on very large datasets
- Needs feature scaling and tuning (C, kernel)
- No direct probabilities

K-Nearest Neighbours

ADVANTAGES

- No training step; very intuitive
- Handles multi-class naturally
- Flexible, non-linear boundaries

DISADVANTAGES

- Slow predictions on big data
- Needs scaling; affected by noise
- Choosing k matters

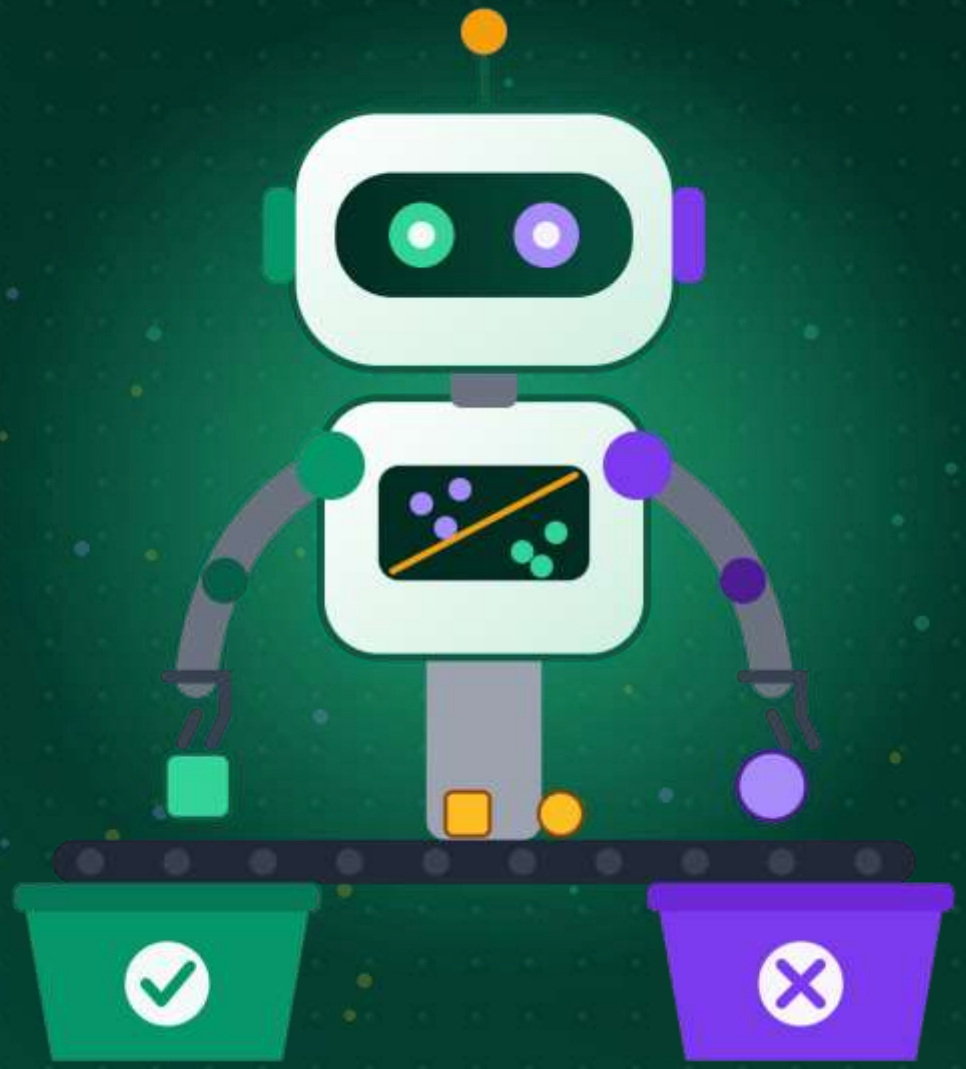
SUMMARY

Which Model When?

	Logistic Regression	SVM	KNN
Boundary	Straight line	Line or curve (kernel)	Any shape (local vote)
Outputs probability	Yes	Not by default	Vote share
Feature scaling	Helpful	Required	Required
Training / prediction	Fast / fast	Slower / fast	None / slow
Our dataset	Grasp success	Terrain, danger zone	Parts sorting
Test accuracy	0.80	0.94 (linear), 0.91 (RBF)	0.77
Use it when	You need a simple, explainable baseline	Clear margin or curved boundaries	Small data, many classes

Thank You

Keep learning, keep building



VSP TECHVERSE
An Institute of AI & Robotics